A Design of Pipelined Carry-dependent Sum Adder With its Self-checking Structure*

Ming Li[1], Shiyi Xu[2], Jialin Cao[1], Feng Ran[1], Shiwei Ma[1]
1. Microelectronic Research & Development Center, Shanghai University, Shanghai 200072 China
liming_21th@yahoo.com
2. School of Computer Science, Shanghai University, Shanghai 200072 China
syxu@fudan.ac.cn

Abstract:

In this paper a pipelined carry-dependent sum adder with the self-checking structure is proposed. The adder includes four 8-bit carry-dependent sum adder (CDSA) , a 4-bit block carry look-ahead unit (BCLU) and a parity checker. The necessary area of the proposed adder is only about 3.85% over the traditional ripple carry adders, while the sum of the traditional adders is delayed by 39.2% with respect to the proposed adder for 32-bit implementation.

Keywords:
Pipelined, Adder, Carry-dependent, Parity, Self-checking,

1. Introduction

Recent advances in integrated circuit design have shown that crosstalk issues in deep submicron can cause severe validation and test problems [1-2].

As the dimensions shrunk, physical problems such as power supply, threshold voltage, short channel effect, gate dielectric leakage, and high-field effects have become more important.

*This work has been supported by National Science Foundation of China (NFSC) under the Grant number 60473033 and Shanghai Leading Academic Discipline Project T0103.

To make things worse, devices are now more sensitive to the external environments than ever, such as radiation effects, temperature variations, and electromagnetic interference [3-5].

For the above reasons, the introduction of self-checking or fault-tolerance into integrated circuits has become extremely important issue in the semiconductor industry nowadays.

Arithmetic units (i.e. adders, ALUs) are some essential components in computers, hence self-checking logic to be embedded in it could be very useful in design of fault tolerant VLSI.

Many different structures can be used to build an adder, such as serial and parallel structures. Ripple carry adder is one of the typical serial adders for which many testing strategies have been reported.

Adders with parallel structures, such as carry look-ahead adders, carry select adders, conditional sum adders and carry skip adders, can be implied in fast field. There are also many testing structures proposed by several researchers [6-8].

A new self-checking code-disjoint almost duplicated fast carry-skip adder has been proposed [9]. Compared to a self-checking duplicated carry look-ahead adder, the maximal delay is reduced by 3% and the necessary area is 33% smaller than before.

A pipelined clock-delayed domino carry

0-7695-2628-4/06 $20.00 © 2006 IEEE
IEEE COMPUTER SOCIETY
Authorized licensed use limited to: INZEDNAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 25,2026 at 13:40:43 UTC from IEEE Xplore. Restrictions apply.

look-ahead adder has also been presented [10]. They compared two pipeline structures (i.e. ADD4$\times$8 and ADD1$\times$32) and demonstrated that the use of clock-delayed domino gates can efficiently reduce the overall critical path delay.

In this paper a pipelined carry-dependent sum adder with the self-checking structure is proposed. We design an adder for the case of 32-bit, which consists of four 8-bit carry-dependent sum adders (CDSA) , a 4-bit block carry look-ahead unit (BCLU) and a parity checker. Because of the use of pipeline structure and carry look-ahead unit, the area and the timing delay of the proposed adder can be reduced for application.

The paper is organized as follows. The carry-dependent sum adder is introduced in Section 2, and the proposed adder is described in Section 3. The experimental results and the comparison between proposed and traditional ripple-carry adder are shown in Section 4. Finally, we give the Conclusions in Section 5.

2. Carry-dependent Sum Adder

Let A and B be the two inputs, S the resulting sum at the outputs of the adder. The two operating numbers add up bit by bit, the ith sum bit s_i depends not only on the inputs a_i and b_i, but also the carry c_{i-1} of the former phase. The binary sum and carry can be expressed by the following equations:

$$s_i = a_i \oplus b_i \oplus c_{i-1}$$

$$c_i = a_i \cdot b_i + (a_i + b_i) \cdot c_{i-1}$$

For the sake of convenience, the function of the adder can be also expressed by the following functions:

$$G_i = a_i \cdot b_i$$

$$T_i = a_i + b_i$$

The above formulas can be expressed by using these functions as follows:

$$s_i = G_i \oplus T_i \oplus c_{i-1}$$

$$c_i = G_i + T_i \cdot c_{i-1}$$

Then the parity of the sum is:

$$\begin{aligned} p_s &= s_{n-1} \oplus \cdots \oplus s_1 \oplus s_0 = \\ &\quad (a_{n-1} \oplus b_{n-1} \oplus c_{n-2}) \oplus \cdots \oplus \\ &(a_1 \oplus b_1 \oplus c_0) \oplus (a_0 \oplus b_0 \oplus c_{in}) \\ &= (a_{n-1} \oplus \cdots \oplus a_1 \oplus a_0) \\ &\oplus (b_{n-1} \oplus \cdots \oplus b_1 \oplus b_0) \\ &\oplus (c_{n-2} \oplus \cdots \oplus c_0 \oplus c_{in}) \\ &= p_A \oplus p_B \oplus p_C \end{aligned}$$

where p_A, p_B are the parities of the two input operating numbers, and p_C are the parity of the carry vector.

By comparing the parity of sum with the parities produced by p_A ,p_B and p_C in the circuits, then we can detect the error in the sum. However, some errors can also occur in the carry logic. In this case, an error in c_{i-1} may also generate errors in s_j and c_j ($j \geqslant i+1$), so it results in double errors in the circuit which will mask both errors by the parity checker.

In solving this issue, the simplest but could be the most expensive way is to duplicate the adders to check the duplicated outputs of the adders by a self-checking two-rail checker, in which the input parities p_A, p_B are produced by an XOR-tree circuits and output parity p_S is produced by another XOR-tree circuits. Another method is to duplicate the carry circuits, one is in the adder, and the other is used to check the parity. However, this method also needs too many areas to be implemented.

Now, there comes another method known as carry-dependent sum adder. In this technique, the sum bit s_i of adder is determined as :

$$s_i = f_i \oplus c_i$$

In above, f_i is the function of a_i, b_i and c_{i-1}.

0-7695-2628-4/06 $20.00 © 2006 IEEE
Authorized licensed use limited to: NATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 25,2026 at 13:40:43 UTC from IEEE Xplore. Restrictions apply.

f_i can be expressed as:

$$f_i = \overline{a_i b_i c_{i-1} + \overline{a_i}\,\overline{b_i}\,\overline{c_{i-1}}} = \overline{G_i c_{i-1} + \overline{T_i}\,\overline{c_{i-1}}}$$

Then, the sum bit can be:

$$s_i = G_i c_{i-1} c_i + \overline{G_i} c_{i-1} \overline{c_i} + T_i \overline{c_{i-1}}\,\overline{c_i} + \overline{T_i}\,\overline{c_{i-1}} c_i$$

The truth table is given in table1.

Table 1 Truth table of carry-dependent sum adder

a_i	b_i	c_{i-1}	s_i	c_i	f_i
0	0	0	0	0	0
0	0	1	1	0	1
0	1	0	1	0	1
0	1	1	0	1	1
1	0	0	1	0	1
1	0	1	0	1	1
1	1	0	0	1	1
1	1	1	1	1	0

Thus, if there is an error occurring in c_i, then the sum bit s_i will also become error. In this case, the even number of errors caused by carry bit will become an odd number of errors. If an error in carry logic causes odd number of burst errors, then the errors can still be detected by the parity checker. Therefore, in case of using carry-dependent sum adder, the error in a carry logic will always result in an odd number of errors in sum and carry bits, and the odd number of burst errors bits can be detected by parity checking.

3. A Proposed Pipeline CDSA

In a ripple-carry adder the carries are propagated step by step through the adder cells. Compared to faster designs the necessary area for a ripple adder is relatively small.

In a carry look-ahead adder the carries are generated from the look-ahead unit, which can work in a high speed but occupies relatively large space in the chip.

For high speed microprocessors, we are interested in 16 or 32-bit of adder, however, the carry look-ahead logic won't be suitable to the circuit since it takes too large area to be embedded in a tiny wafer. Actually, in an alternative way, a carry look-ahead adder can easily be designed as a pipeline [10] so that the area could no longer be the problematic. In this paper, we design the carry look-ahead adder in pipeline structure, and also implement the self-checking function of the adder by using the method of carry-dependent sum. The configurations are designed and simulated to measure the critical-path latency and gate count.

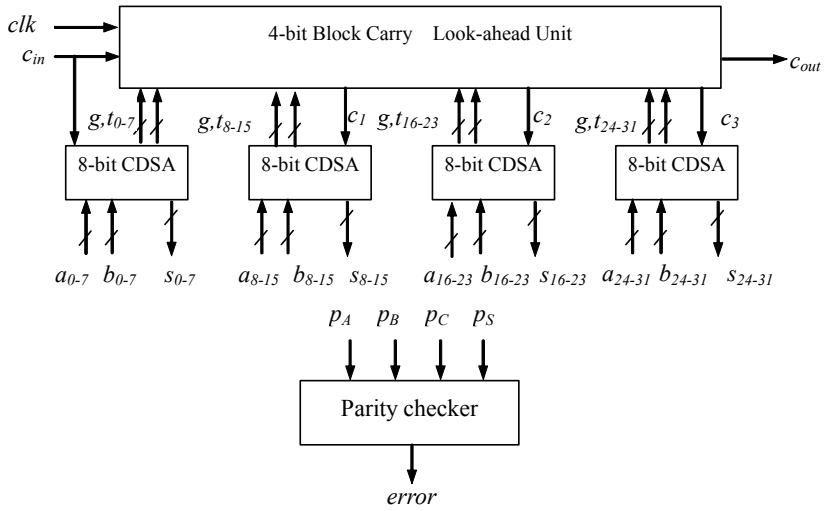


Figure 1. The structure of the proposed pipeline carry-dependent sum adder

0-7695-2628-4/06 $20.00 © 2006 IEEE
Authorized licensed use limited to: NATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 25,2026 at 13:40:43 UTC from IEEE Xplore. Restrictions apply.

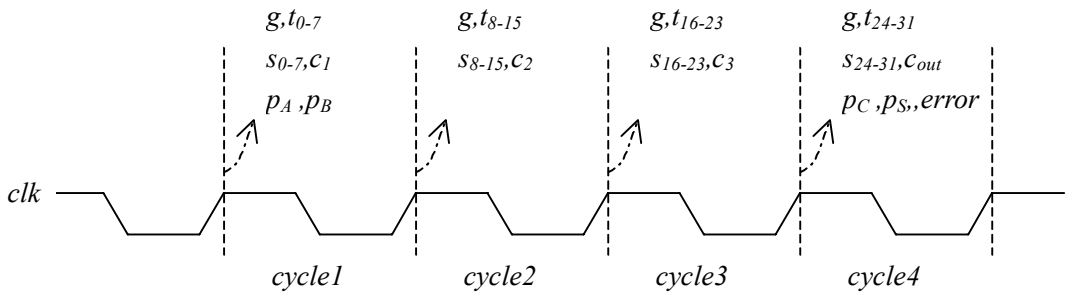


Figure 2. The parity and error signal generation of the pipeline process

In this design, a 32-bit carry look-ahead adder is implemented including block carry look-ahead generation with 4-bit blocks and its parity checking units. The block diagram of this configuration is illustrated in Figure 1. This adder uses four 8-bit carry-dependent sum adder blocks with a 4-bit block carry look-ahead unit. A CDSA is a carry-dependent sum adder in which the sum and transmitting signals are produced. In fact, the carry-out signal is not produced by the adder but the carry look-ahead unit. The pipeline design of the adder is shown in Figure 2. The whole adder completes its calculation of the sum and the error checking within four clock cycles. As soon as the adder generates four 8-bit sum and a carry, the parity checker directly offers the parity of the sum and is able to detect the errors, if any.

4. Experimental Results and Comparison

Experimental results are presented here in this section for the proposed pipelined carry-dependent sum adder. Both adders are modeled in an RTL description in verilog HDL by the synplifypro8.0 CAD tool, mapped to the library of "Actel ProASIC3" and optimized. Not only is the 4×8 pipelined adder considered, but also 4-bit, 8-bit, 16-bit and 32-bit adders are addressed.

In table 2 the delays and the necessary area for pipelined carry-dependent sum adder and ripple-carry adders are presented for comparison.

We can see from table 2 that the necessary area of the proposed pipelined carry-dependent sum adders is only about 3.85% over the corresponding ripple carry adders for 32-bit.

The sums of the ripple carry adders for 4-bit, 8-bit, 16-bit and 32-bit are delayed by 17.4%, 22.1%, 30.9% and 39.2% with respect to the proposed adders, respectively.

The schematic graph of the 32-bit pipeline carry-dependent sum adder synthesized by synplifypro8.0 is shown in Figure 3 in the last of the paper..

Table 2 Pipelined carry-dependent sum adders and ripple-carry adders

Adders	Pipeline carry-dependent sum adders		Ripple-carry adders	
	Delay(ns)	Gates	Delay(ns)	Gates
4-bit	7.237	39	8.498	33
8-bit	10.612	103	12.961	91
16bits	14.375	238	18.824	223
32-bit	19.352	405	26.943	390

5. Conclusions

In this paper, a pipelined carry-dependent sum adder with its self-checking structure was proposed. We have designed the adders of 4-bit, 8-bit, 16-bit and 32-bit, but demonstrated the

0-7695-2628-4/06 $20.00 © 2006 IEEE
Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 25,2026 at 13:40:43 UTC from IEEE Xplore. Restrictions apply.

scheme for the adder with 32-bit only for simplicity, which consists of four 8-bit carry-dependent sum adders (CDSA), a 4-bit block carry look-ahead unit (BCLU) and a parity checker. It has shown that both the area and the delay of the proposed adder can be reduced in real application by using the pipeline structure and carry look-ahead unit.

The exact results have proved that the necessary area of the proposed adders is only about 3.85% over the traditional ripple carry adders, while the implementation of sum of the traditional adders delays longer by 39.2% comparing to that of the proposed adder with 32-bit. Therefore, it is quite evident that the area and delay of the adders can be further reduced in case of using more advanced microelectronic technology, which will be our next objective.

Acknowledgments

The authors will express their sincere gratefulness to the Power Station of Shanghai Key Laboratory in which all the experiments are completed.

References

[1] Cristian Constantinescu, "Impact of Deep Submicron Technology on Dependability of VLSI Circuits", in *Proceedings of the International Conference on Dependable Systems and Networks* (DSN'02),2002.

[2] Hamidreza Hashempour, Yong-Bin Kim, Naphill Park, "A Test- Vector Generation Methodology for Crosstalk Noise Faults" , in *Proceedings of the 17th IEEE International Symposium on Defect and Fault Tolerance in VLSI Systems* (DFT'02) ,2002.

[3] Arvind Kumar and Sandip Tiwari, "Testing and Defect Tolerance: A Rent's Rule Based Analysis and Implications on Nano-electronics", in *Proceedings of the 19th IEEE International Symposium on Defect and Fault Tolerance in VLSI Systems* (DFT'04), 2004.

[4] L. Anghel, M. Nicolaidis, and I. Alzaher-Noufal, "Self-Checking Circuits versus Realistic Faults in Very Deep Submicron," in *18th VLSI Test Symposium*, pp. 55–63, 2000.

[5] C. Metra, M. Favalli, and B. Ricco, "On-line Detection of Logic Errors due to Crosstalk, Delay and Transient Faults," in *Proceedings of ITC*, pp. 524–533.

[6] M. NlCOLAlDlS, "Efficient Implementations of Self-checking Adders and ALUs", in *Symposium on Fault-Tolerant Computing*, pp.556-595, 1993.

[7] Jin-Fu Li and Chih-Chiang Hsu, "Efficient Testing Methodologies for Conditional Sum Adders", in *Proceedings of the 13th Asian Test Symposium* (ATS 2004), 2004.

[8] D. Marienfeld, V. Ocheretnij,M. Gössel, "Partially duplicated Code-disjoint Carry-skip Adder", in *Proceedings of the 17th IEEE International Symposium on Defect and Fault Tolerance in VLSI Systems* (DFT'02), 2002.

[9] V. Ocheretnij, M. Gössel, and E. S. Sogomonyan, "Code-disjoint Carry-dependent Sum Adder with Partial Look-ahead," in *Proceedings of 7th International On-Line Testing Workshop*, pp. 147–152, 2001.

[10] Bhushan A.Shinkre and James E. Stine, "A Pipelined Clock-Delayed Domino Carry-Lookahead Adder", in *GLSVLSI'03*, April 28-29,2003.

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 25,2026 at 13:40:43 UTC from IEEE Xplore. Restrictions apply.

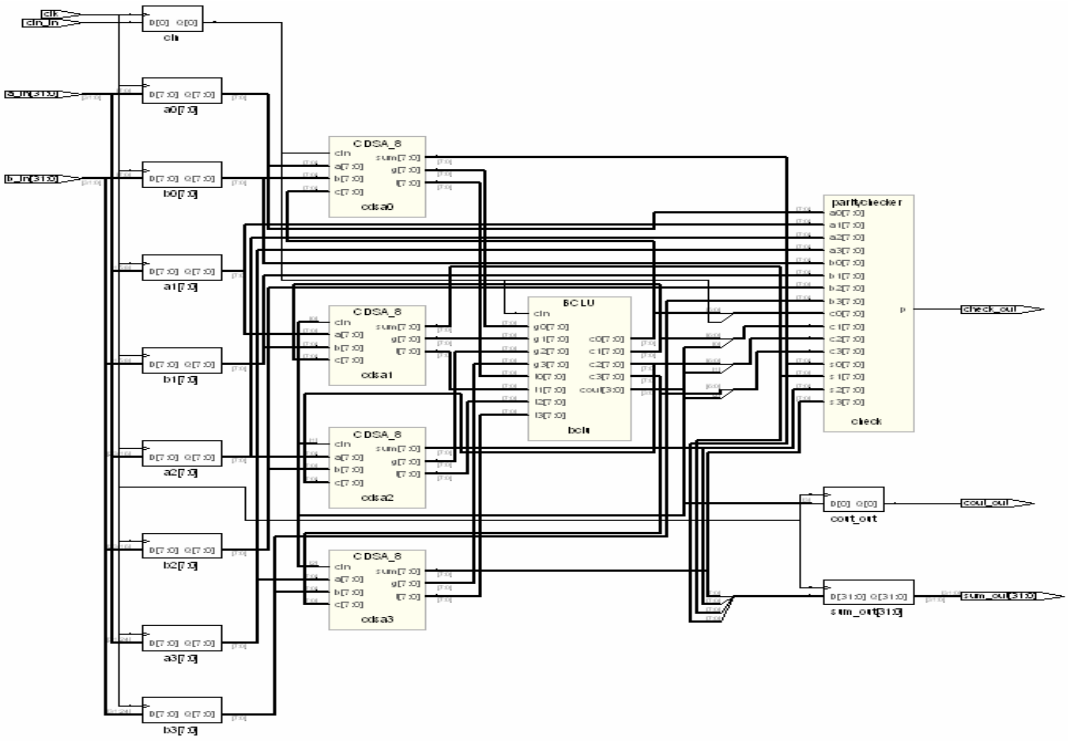


Figure 3. The 32-bit pipelined carry-dependent sum adder

0-7695-2628-4/06 $20.00 © 2006 IEEE
Authorized licensed use limited to: NATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 25,2026 at 13:40:43 UTC from IEEE Xplore. Restrictions apply.